

Coding & Solution

Date	28 September 2026
Team ID	6ab4d2876603573ccfb56348
Project Name	AI WEATHERWISE
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/dharshinibca2024/Weather_Wise_AI.git
Programming Language(s)	JavaScript (Node.js)
Framework(s) Used	Express.js (Backend), MongoDB / Mongoose (Database)

Key Implemented	Features	• User Authentication (JWT based Register & Login) • Real-time Weather Data retrieval via External API Integration • AI-generated Weather Summaries and Recommendations • Saved Locations management (CRUD operations via MongoDB) • Error handling & Input validation middleware
Pending / Incomplete Features		• Advanced interactive maps visualization • Push notifications for extreme weather alerts
Setup / Run Instructions		1. Clone the repository and navigate to the project directory: <code>cd Backend</code> 2. Install dependencies: <code>npm install</code> 3. Configure Environment Variables: Create a <code>.env</code> file with <code>PORT=5500</code>, <code>MONGO_URI</code>, <code>JWT_SECRET</code>, and external API keys. 4. Start the application: Run <code>npm start</code> or <code>node src/server.js</code>. 5. Test APIs: Import <code>postman_collection.json</code> into Postman to test endpoints.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes

3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

- ☐ The backend architecture strictly adheres to the MVC (Model-View-Controller) structure, separating controllers (authController, weatherController, aiController), routes, models (User, Location), and middleware (authMiddleware). Environment configurations are safely isolated in .env, and database connections are handled asynchronously with fallback error logs.